

Pack 26 (Web + API + Shared) — Ed25519 Receipt Signing + Server Key Management for Non-Repudiation (real code)

RemoteDesk · Pack 26 (Web + API + Shared) — Ed25519 Receipt Signing + Server Key Management for Non-Repudiation (real code)

Codex / Claude Code handoff. Direct continuation of Packs 22 → 25. So far a consent receipt proves INTERNAL consistency (its own hash chain). This slice adds cryptographic NON-REPUDIATION: the server signs each verified receipt's root hash with an Ed25519 key, publishes the public key, and anyone can verify the signature offline. The signature binds the org + receipt + root hash so a receipt can't be re-attributed or forged. Build order: shared signature contract + verifier → server keypair management → sign-on-store → public-key + verify endpoints → web badge → tests.

Scope: detached Ed25519 signatures over a canonical receipt digest, server-held private key (never leaves the server, never logged), published public key for offline verification. Reuses Pack 24 verification (a receipt is signed ONLY after it validates). **Private key never in responses/logs. Signature covers org+receiptId+rootHash+issuedAt. Verification is content-free + offline. No new heavy crypto deps beyond Node/WebCrypto Ed25519.**

Tag legend: **[NEW]** = create file. **[MODIFY]** = patch existing file by hand (merge, don't overwrite).

Files created / modified / not touched

CREATED [NEW]	
packages/shared/src/permissions/receiptSignature.ts	
apps/api/prisma/migrations/0026_signing_keys/migration.sql	
apps/api/src/lib/ed25519.ts	
apps/api/src/repositories/signingKeyRepository.ts	
apps/api/src/services/admin/receiptSigner.service.ts	
apps/web/app/dashboard/receipts/SignatureBadge.tsx	
tests/admin/receiptSignature.test.ts	
tests/admin/ed25519.test.ts	
tests/admin/receiptSigner.service.test.ts	
MODIFIED [MODIFY]	
packages/shared/src/permissions/index.ts	(export receiptSignature)
packages/shared/src/permissions/consentReceipt.ts	(optional signature field on ConsentReceipt)
apps/api/prisma/schema.prisma	(SigningKey model + receipt signature columns)
apps/api/src/services/admin/receiptVerifier.service.ts	(sign after verify on store)
apps/api/src/routes/deviceAdmin.routes.ts	(GET signing pubkey; verify returns signature state)
apps/web/app/dashboard/receipts/page.tsx	(render SignatureBadge)
NOT TOUCHED (intentionally)	
apps/desktop/**	(desktop builds the receipt; server signs it)
apps/api/src/lib/nodeSha256.ts	(Pack 24 hash unchanged)

Part A — Shared: signature contract + offline verifier

FILE: packages/shared/src/permissions/receiptSignature.ts

```
import type { ConsentReceipt } from './consentReceipt';

/**
 * Detached Ed25519 signature over a CANONICAL receipt digest. The signature binds
 * the org, receiptId, rootHash, and issuedAt, so a valid receipt can't be silently
 * re-attributed to another org or have its root swapped. Verification is offline:
 * given the receipt + signature + the org's published public key, anyone can check
 * it with no server call.
 */
export interface ReceiptSignature {
  /** Key id of the signing key (rotation-aware). */
  keyId: string;
  /** Ed25519 signature, base64url. */
  signature: string;
  /** Algorithm tag for forward-compat. */
  alg: 'ed25519';
  signedAt: string;
}

/** A published signing public key. */
export interface SigningPublicKey {
  keyId: string;
  /** Raw Ed25519 public key, base64url (32 bytes). */
  publicKey: string;
  alg: 'ed25519';
  createdAt: string;
  /** Set when rotated out; verification of old signatures still works. */
  retiredAt: string | null;
}

/**
 * Canonical signing preimage. ORDER-STABLE + content-free: org + receiptId +
 * rootHash + issuedAt only. Never includes summary internals or line items (the
 * rootHash already commits to those via the Pack 22 chain).
 */
export function signingPreimage(orgId: string, receipt: Pick<ConsentReceipt, 'receiptId' | 'rootHash' | 'issuedAt'>):
string {
  return `remotedesk-receipt-v1|${orgId}|${receipt.receiptId}|${receipt.rootHash}|${receipt.issuedAt}`;
}

/** Injected verify fn so shared stays runtime-agnostic (Node/WebCrypto supply it). */
export type Ed25519Verify = (publicKeyB64Url: string, message: string, signatureB64Url: string) => Promise<boolean>;

/**
 * Offline signature verification. Confirms the detached signature over the
 * canonical preimage using the published public key. Returns false on any
 * mismatch (fail-closed).
 */
export async function verifyReceiptSignature(
  orgId: string,
```

```

    receipt: ConsentReceipt,
    signature: ReceiptSignature,
    publicKey: SigningPublicKey,
    ed25519Verify: Ed25519Verify,
  ): Promise<boolean> {
    if (signature.alg !== 'ed25519' || publicKey.alg !== 'ed25519') return false;
    if (signature.keyId !== publicKey.keyId) return false;
    const message = signingPreimage(orgId, receipt);
    try {
      return await ed25519Verify(publicKey.publicKey, message, signature.signature);
    } catch {
      return false;
    }
  }
}

```

FILE: packages/shared/src/permissions/consentReceipt.ts [MODIFY]

```

// ...existing Pack 22 contents...
import type { ReceiptSignature } from './receiptSignature';

// ---- MODIFY: add an OPTIONAL signature field to ConsentReceipt ----
// export interface ConsentReceipt {
//   ...existing fields (receiptId, summary, lineItems, rootHash, issuedAt, version)...
//   /** Server-applied Ed25519 signature; absent until the server signs it. */
//   signature?: ReceiptSignature;
// }
//
// NOTE: the signing preimage uses receiptId + rootHash + issuedAt only, so adding
// the signature object does NOT change what is signed (no chicken-and-egg).

```

FILE: packages/shared/src/permissions/index.ts [MODIFY]

```

// ...existing exports through Pack 25...
export * from './receiptSignature'; // ADD

```

Part B — API: Prisma schema + migration

FILE: apps/api/prisma/schema.prisma [MODIFY]

```

// ---- ADD: server signing keys (private key stored server-side; NEVER returned) ----
model SigningKey {
  id          String   @id @default(cuid())
  keyId       String   @unique
  orgId       String?   // null = workspace-global signing key
  alg         String   @default("ed25519")
  publicKey   String    // base64url raw public key (publishable)
  privateKey  String    // base64url raw private key (SECRET; never serialized to clients)
  createdAt   DateTime @default(now())
  retiredAt   DateTime?

  @@index([orgId])
}

// ---- MODIFY ConsentReceiptRecord: store the signature alongside the receipt ----

```

```
// model ConsentReceiptRecord {
//   ...existing Pack 23 fields...
//   signatureKeyId String?
//   signature      String? // base64url Ed25519 signature
//   signedAt       DateTime?
// }
```

FILE: apps/api/prisma/migrations/0026_signing_keys/migration.sql

```
-- SigningKey (private key column is SECRET; never selected into API responses)
CREATE TABLE "SigningKey" (
  "id"          TEXT PRIMARY KEY,
  "keyId"       TEXT NOT NULL UNIQUE,
  "orgId"       TEXT,
  "alg"         TEXT NOT NULL DEFAULT 'ed25519',
  "publicKey"   TEXT NOT NULL,
  "privateKey"  TEXT NOT NULL,
  "createdAt"   TIMESTAMP NOT NULL DEFAULT now(),
  "retiredAt"   TIMESTAMP
);
CREATE INDEX "SigningKey_orgId_idx" ON "SigningKey"("orgId");

-- ConsentReceiptRecord signature columns (additive)
ALTER TABLE "ConsentReceiptRecord" ADD COLUMN IF NOT EXISTS "signatureKeyId" TEXT;
ALTER TABLE "ConsentReceiptRecord" ADD COLUMN IF NOT EXISTS "signature" TEXT;
ALTER TABLE "ConsentReceiptRecord" ADD COLUMN IF NOT EXISTS "signedAt" TIMESTAMP;
```

Part C — API: Ed25519 primitives

FILE: apps/api/src/lib/ed25519.ts

```
import { generateKeyPairSync, sign as nodeSign, verify as nodeVerify, createPublicKey, createPrivateKey } from
'node:crypto';

/**
 * Ed25519 helpers using Node's built-in crypto (no external deps). Keys are stored
 * as base64url RAW 32-byte values; we wrap/unwrap to DER for Node's KeyObject API.
 * The private key is handled ONLY here and in the signer service; never logged,
 * never serialized to a client.
 */

const RAW_PUB_PREFIX = Buffer.from('302a300506032b6570032100', 'hex'); // SPKI Ed25519 header
const RAW_PRIV_PREFIX = Buffer.from('302e020100300506032b657004220420', 'hex'); // PKCS8 Ed25519 header

export function generateEd25519(): { publicKey: string; privateKey: string } {
  const { publicKey, privateKey } = generateKeyPairSync('ed25519');
  const pubRaw = publicKey.export({ type: 'spki', format: 'der' }).subarray(RAW_PUB_PREFIX.length);
  const privRaw = privateKey.export({ type: 'pkcs8', format: 'der' }).subarray(RAW_PRIV_PREFIX.length);
  return { publicKey: b64url(pubRaw), privateKey: b64url(privRaw) };
}

export function signEd25519(privateKeyB64Url: string, message: string): string {
  const der = Buffer.concat([RAW_PRIV_PREFIX, fromB64url(privateKeyB64Url)]);
  const key = createPrivateKey({ key: der, format: 'der', type: 'pkcs8' });
  const sig = nodeSign(null, Buffer.from(message, 'utf8'), key);
```

```

    return b64url(sig);
  }

export async function verifyEd25519(publicKeyB64Url: string, message: string, signatureB64Url: string): Promise<boolean>
{
  const der = Buffer.concat([RAW_PUB_PREFIX, fromB64url(publicKeyB64Url)]);
  const key = createPublicKey({ key: der, format: 'der', type: 'spki' });
  return nodeVerify(null, Buffer.from(message, 'utf8'), key, fromB64url(signatureB64Url));
}

function b64url(buf: Buffer): string {
  return buf.toString('base64').replace(/#/g, '%').replace(/%/g, '%').replace(/=+$/, '');
}

function fromB64url(s: string): Buffer {
  return Buffer.from(s.replace(/-/g, '+').replace(/_/g, '/'), 'base64');
}

```

FILE: apps/api/src/repositories/signingKeyRepository.ts

```

import type { SigningPublicKey } from '@remotedesk/shared';
import { prisma } from '../db/prisma';
import { generateEd25519 } from '../lib/ed25519';

/**
 * Manages signing keys. CRITICAL: privateKey is selected ONLY by signingPrivate()
 * for use inside the signer service. The public listing/get NEVER includes it.
 */
export const signingKeyRepository = {
  /** Public key for publishing (no private material). */
  async publicKey(orgId: string | null): Promise<SigningPublicKey | null> {
    const k = await prisma.signingKey.findFirst({
      where: { orgId, retiredAt: null },
      orderBy: { createdAt: 'desc' },
      select: { keyId: true, publicKey: true, alg: true, createdAt: true, retiredAt: true },
    });
    return k ? { keyId: k.keyId, publicKey: k.publicKey, alg: 'ed25519', createdAt: k.createdAt.toISOString(), retiredAt: k.retiredAt ? k.retiredAt.toISOString() : null } : null;
  },

  /** Internal-only: private key for signing. NEVER expose via a route. */
  async signingPrivate(orgId: string | null): Promise<{ keyId: string; privateKey: string } | null> {
    const k = await prisma.signingKey.findFirst({
      where: { orgId, retiredAt: null },
      orderBy: { createdAt: 'desc' },
      select: { keyId: true, privateKey: true },
    });
    return k ?? null;
  },

  /** Lazily create the active signing key for an org if none exists. */
  async ensureKey(orgId: string | null): Promise<{ keyId: string; privateKey: string }> {
    const existing = await this.signingPrivate(orgId);
    if (existing) return existing;
    const { publicKey, privateKey } = generateEd25519();
    const keyId = `k_${Date.now().toString(36)}`;
    await prisma.signingKey.create({ data: { keyId, orgId, alg: 'ed25519', publicKey, privateKey } });
  }
};

```

```

    return { keyId, privateKey };
  },

  /** Look up a public key by id (for verifying old signatures after rotation). */
  async publicKeyById(keyId: string): Promise<SigningPublicKey | null> {
    const k = await prisma.signingKey.findUnique({
      where: { keyId },
      select: { keyId: true, publicKey: true, alg: true, createdAt: true, retiredAt: true },
    });
    return k ? { keyId: k.keyId, publicKey: k.publicKey, alg: 'ed25519', createdAt: k.createdAt.toISOString(), retiredAt:
k.retiredAt ? k.retiredAt.toISOString() : null } : null;
  },
};

```

Part D — API: signer service + verify integration

FILE: apps/api/src/services/admin/receiptSigner.service.ts

```

import {
  signingPreimage,
  verifyReceiptSignature,
  type ConsentReceipt,
  type ReceiptSignature,
  type SigningPublicKey,
} from '@remotedesk/shared';
import { signEd25519, verifyEd25519 } from '../../lib/ed25519';
import { signingKeyRepository } from '../../repositories/signingKeyRepository';

/**
 * Signs verified receipts + verifies signatures. The private key never leaves
 * this service (loaded, used to sign, discarded). Signing happens ONLY after the
 * Pack 24 verifier confirms the receipt is valid + content-free.
 */
export const receiptSignerService = {
  /** Produce a detached signature over the canonical preimage. */
  async sign(orgId: string, receipt: ConsentReceipt): Promise<ReceiptSignature> {
    const key = await signingKeyRepository.ensureKey(orgId);
    const message = signingPreimage(orgId, receipt);
    const signature = signEd25519(key.privateKey, message);
    return { keyId: key.keyId, signature, alg: 'ed25519', signedAt: new Date().toISOString() };
  },

  /** Publish the org's current public key (no private material). */
  publicKey(orgId: string): Promise<SigningPublicKey | null> {
    return signingKeyRepository.publicKey(orgId);
  },

  /**
   * Verify a receipt's signature server-side (also possible fully offline by a
   * third party using the published public key + shared verifier).
   */
  async verify(orgId: string, receipt: ConsentReceipt, signature: ReceiptSignature): Promise<boolean> {
    const pub = await signingKeyRepository.publicKeyById(signature.keyId);
    if (!pub) return false;
    return verifyReceiptSignature(orgId, receipt, signature, pub, verifyEd25519);
  }
};

```

```
},  
};
```

FILE: apps/api/src/services/admin/receiptVerifier.service.ts [MODIFY]

```
// ...existing Pack 24 imports...  
import { receiptSignerService } from '../receiptSigner.service';  
  
// ---- MODIFY storeVerified: after validation passes, SIGN then persist with sig ----  
// async storeVerified(orgId, receipt) {  
//   const result = await validateReceipt(receipt, nodeSha256Hex);  
//   if (!result.valid) { /* Pack 25 audit + throw RECEIPT_INVALID */ }  
//   const signature = await receiptSignerService.sign(orgId, receipt); // ADD  
//   const signed = { ...receipt, signature }; // ADD  
//   await consentReceiptRepository.store(orgId, signed.summary.deviceId, signed); // store signed copy  
//   return result;  
// }  
//  
// NOTE: consentReceiptRepository.store should persist signatureKeyId/signature/  
// signedAt from receipt.signature (Part B columns). The rootHash signed is the  
// SAME one validateReceipt confirmed, so signature + chain agree.
```

Part E — API: routes

FILE: apps/api/src/routes/deviceAdmin.routes.ts [MODIFY]

```
// ...existing Pack 23/24/25 imports...  
import { receiptSignerService } from '../services/admin/receiptSigner.service';  
import { consentReceiptRepository } from '../repositories/consentReceiptRepository';  
  
// ---- ADD: publish the org signing public key (NO private material) ----  
// .get('/receipts/signing-key', requireAuth, orgContext, h(async (req, res) => {  
//   const pub = await receiptSignerService.publicKey((req as OrgRequest).orgId);  
//   res.json(pub ?? { keyId: null }); // null when no receipt has been signed yet  
// })))  
  
// ---- MODIFY /receipts/:id/verify: also report signature validity ----  
// .get('/receipts/:receiptId/verify', requireAuth, orgContext, h(async (req, res) => {  
//   const orgId = (req as OrgRequest).orgId;  
//   const chain = await receiptVerifierService.verifyStored(orgId, req.params.receiptId);  
//   const receipt = await consentReceiptRepository.getJson(orgId, req.params.receiptId);  
//   const signatureValid = receipt?.signature  
//     ? await receiptSignerService.verify(orgId, receipt, receipt.signature)  
//     : false;  
//   res.json({ ...chain, signatureValid, signed: !!receipt?.signature });  
// })))  
//  
// SECURITY: never add a route that returns SigningKey.privateKey.
```

Part F — Web: signature badge

FILE: apps/web/app/dashboard/receipts/SignatureBadge.tsx

```

'use client';

import React from 'react';

/**
 * Shows whether a receipt is cryptographically signed + (after verify) whether the
 * signature checks out. Distinct from the Pack 24 chain-intact badge: this is
 * non-repudiation (the SERVER attests it), not just internal consistency.
 */
export function SignatureBadge({
  signed,
  signatureValid,
}): {
  signed: boolean;
  signatureValid: boolean | null; // null = not yet verified
}) {
  if (!signed) return <span className="sig-badge sig-badge--unsigned">unsigned</span>;
  if (signatureValid === null) return <span className="sig-badge sig-badge--signed">signed</span>;
  return signatureValid
    ? <span className="sig-badge sig-badge--valid">✓ signature valid</span>
    : <span className="sig-badge sig-badge--invalid">✗ signature invalid</span>;
}

```

FILE: apps/web/app/dashboard/receipts/page.tsx [MODIFY]

```

// ...existing Pack 23/24 imports...
import { SignatureBadge } from './SignatureBadge';

// ---- MODIFY the verify handler to capture signature state from the response ----
// const v = await api.get<{ valid: boolean; signatureValid: boolean; signed: boolean }>(
//   `/devices/receipts/${receiptId}/verify`,
// );
// setVerified((prev) => ({ ...prev, [receiptId]: v }));

// ---- ADD a column rendering the SignatureBadge ----
// <td>
//   <SignatureBadge
//     signed={verified[r.receiptId]?.signed ?? false}
//     signatureValid={verified[r.receiptId] ? verified[r.receiptId].signatureValid : null}
//   />
// </td>
//
// Optional: a link to GET /devices/receipts/signing-key so auditors can grab the
// public key for fully offline verification with the shared verifyReceiptSignature.

```

Part G — Tests

FILE: tests/admin/ed25519.test.ts

```

import { describe, it, expect } from 'vitest';
import { generateEd25519, signEd25519, verifyEd25519 } from '../../apps/api/src/lib/ed25519';

describe('ed25519 primitives', () => {
  it('round-trips sign + verify', async () => {

```



```

    const { publicKey, privateKey } = generateEd25519();
    const msg = 'remotedesk-receipt-v1|org|r1|abc|2026-06-15T10:00:00.000Z';
    const sig = signEd25519(privateKey, msg);
    expect(await verifyEd25519(publicKey, msg, sig)).toBe(true);
  });

  it('rejects a tampered message', async () => {
    const { publicKey, privateKey } = generateEd25519();
    const sig = signEd25519(privateKey, 'original');
    expect(await verifyEd25519(publicKey, 'tampered', sig)).toBe(false);
  });

  it('rejects a signature from a different key', async () => {
    const a = generateEd25519();
    const b = generateEd25519();
    const sig = signEd25519(a.privateKey, 'msg');
    expect(await verifyEd25519(b.publicKey, 'msg', sig)).toBe(false);
  });

  it('produces base64url keys (no +/=)', () => {
    const { publicKey, privateKey } = generateEd25519();
    expect(publicKey).not.toMatch(/[+|=]/);
    expect(privateKey).not.toMatch(/[+|=]/);
  });
});

```

FILE: tests/admin/receiptSignature.test.ts

```

import { describe, it, expect } from 'vitest';
import { signingPreimage, verifyReceiptSignature, type ReceiptSignature, type SigningPublicKey } from
 '@remotedesk/shared';
import { generateEd25519, signEd25519, verifyEd25519 } from '../../apps/api/src/lib/ed25519';
import type { ConsentReceipt } from '@remotedesk/shared';

const receipt = {
  receiptId: 'r1', rootHash: 'abc123', issuedAt: '2026-06-15T10:00:00.000Z',
  summary: { deviceId: 'd1' } as any, lineItems: [], version: 1,
} as ConsentReceipt;

describe('receiptSignature (shared)', () => {
  it('canonical preimage binds org + receiptId + rootHash + issuedAt', () => {
    expect(signingPreimage('org1', receipt)).toBe('remotedesk-receipt-v1|org1|r1|abc123|2026-06-15T10:00:00.000Z');
  });

  it('verifies a valid signature offline', async () => {
    const { publicKey, privateKey } = generateEd25519();
    const sig: ReceiptSignature = { keyId: 'k1', signature: signEd25519(privateKey, signingPreimage('org1', receipt)),
alg: 'ed25519', signedAt: 'now' };
    const pub: SigningPublicKey = { keyId: 'k1', publicKey, alg: 'ed25519', createdAt: 'now', retiredAt: null };
    expect(await verifyReceiptSignature('org1', receipt, sig, pub, verifyEd25519)).toBe(true);
  });

  it('fails if the org differs (re-attribution attempt)', async () => {
    const { publicKey, privateKey } = generateEd25519();
    const sig: ReceiptSignature = { keyId: 'k1', signature: signEd25519(privateKey, signingPreimage('org1', receipt)),
alg: 'ed25519', signedAt: 'now' };

```

```

    const pub: SigningPublicKey = { keyId: 'k1', publicKey, alg: 'ed25519', createdAt: 'now', retiredAt: null };
    expect(await verifyReceiptSignature('org2', receipt, sig, pub, verifyEd25519)).toBe(false);
  });

  it('fails on a keyId mismatch', async () => {
    const { publicKey, privateKey } = generateEd25519();
    const sig: ReceiptSignature = { keyId: 'k1', signature: signEd25519(privateKey, signingPreimage('org1', receipt)),
    alg: 'ed25519', signedAt: 'now' };
    const pub: SigningPublicKey = { keyId: 'k2', publicKey, alg: 'ed25519', createdAt: 'now', retiredAt: null };
    expect(await verifyReceiptSignature('org1', receipt, sig, pub, verifyEd25519)).toBe(false);
  });

  it('fails if the rootHash was swapped', async () => {
    const { publicKey, privateKey } = generateEd25519();
    const sig: ReceiptSignature = { keyId: 'k1', signature: signEd25519(privateKey, signingPreimage('org1', receipt)),
    alg: 'ed25519', signedAt: 'now' };
    const pub: SigningPublicKey = { keyId: 'k1', publicKey, alg: 'ed25519', createdAt: 'now', retiredAt: null };
    const tampered = { ...receipt, rootHash: 'deadbeef' };
    expect(await verifyReceiptSignature('org1', tampered, sig, pub, verifyEd25519)).toBe(false);
  });
});

```

FILE: tests/admin/receiptSigner.service.test.ts

```

import { describe, it, expect, vi, beforeEach } from 'vitest';
import { generateEd25519 } from '../../apps/api/src/lib/ed25519';

const kp = generateEd25519();
const ensureKey = vi.fn().mockResolvedValue({ keyId: 'k1', privateKey: kp.privateKey });
const publicKeyById = vi.fn().mockResolvedValue({ keyId: 'k1', publicKey: kp.publicKey, alg: 'ed25519', createdAt: 'now', retiredAt: null });
vi.mock('../../apps/api/src/repositories/signingKeyRepository', () => ({
  signingKeyRepository: {
    ensureKey: (...a: unknown[]) => ensureKey(...a),
    publicKeyById: (...a: unknown[]) => publicKeyById(...a),
    publicKey: vi.fn(),
  },
}));

import { receiptSignerService } from '../../apps/api/src/services/admin/receiptSigner.service';
import type { ConsentReceipt } from '@remotedesk/shared';

const receipt = {
  receiptId: 'r1', rootHash: 'abc123', issuedAt: '2026-06-15T10:00:00.000Z',
  summary: { deviceId: 'd1' } as any, lineItems: [], version: 1,
} as ConsentReceipt;

describe('receiptSignerService', () => {
  beforeEach(() => { ensureKey.mockClear(); });

  it('signs then verifies its own signature', async () => {
    const sig = await receiptSignerService.sign('org1', receipt);
    expect(sig.alg).toBe('ed25519');
    expect(sig.keyId).toBe('k1');
    expect(await receiptSignerService.verify('org1', receipt, sig)).toBe(true);
  });
});

```

```

it('verification fails for a different org', async () => {
  const sig = await receiptSignerService.sign('org1', receipt);
  expect(await receiptSignerService.verify('org2', receipt, sig)).toBe(false);
});

it('never exposes the private key on the signature', async () => {
  const sig = await receiptSignerService.sign('org1', receipt);
  expect(JSON.stringify(sig)).not.toContain(kp.privateKey);
});
});

```

Part H — Reports

FILE: [generated-pack26-receipt-signing-merge-summary.md](#)

Pack 26 — Ed25519 Receipt Signing · Merge Summary

What this slice does

Adds cryptographic non-repudiation to Pack 22 consent receipts. After the Pack 24 verifier confirms a receipt is valid + content-free, the server signs its canonical digest (org + receiptId + rootHash + issuedAt) with an Ed25519 key, stores the signature, and publishes the public key. Anyone can verify offline with the shared verifier — no server call, no private material.

Files created (9)

- packages/shared/src/permissions/receiptSignature.ts
- apps/api/prisma/migrations/0026_signing_keys/migration.sql
- apps/api/src/lib/ed25519.ts
- apps/api/src/repositories/signingKeyRepository.ts
- apps/api/src/services/admin/receiptSigner.service.ts
- apps/web/app/dashboard/receipts/SignatureBadge.tsx
- tests/admin/receiptSignature.test.ts
- tests/admin/ed25519.test.ts
- tests/admin/receiptSigner.service.test.ts

Files modified (6)

- packages/shared/src/permissions/index.ts — export receiptSignature.
- packages/shared/src/permissions/consentReceipt.ts — optional signature field.
- apps/api/prisma/schema.prisma — SigningKey model + receipt signature columns.
- apps/api/src/services/admin/receiptVerifier.service.ts — sign-after-verify on store.
- apps/api/src/routes/deviceAdmin.routes.ts — GET signing-key; verify reports signatureValid.
- apps/web/app/dashboard/receipts/page.tsx — render SignatureBadge.

Files intentionally NOT touched

- apps/desktop/** — desktop builds the receipt; the server signs it.
- apps/api/src/lib/nodeSha256.ts — Pack 24 hashing unchanged.

SECURITY (read this)

- SigningKey.privateKey is SECRET: selected only by signingKeyRepository.signingPrivate/ensureKey, used only inside receiptSignerService, NEVER returned by a route or logged.
- The /receipts/signing-key route returns ONLY the public key.
- The signing preimage is content-free (ids + rootHash + issuedAt); the rootHash already commits to the timeline.

Remaining manual wiring risks

1. consentReceiptRepository.store must persist signatureKeyId/signature/signedAt from receipt.signature.
2. storeVerified must sign AFTER validateReceipt passes (order matters).
3. Add NO route exposing privateKey; review the SigningKey select clauses.
4. Key rotation: retiring a key keeps publicKeyById working for old signatures.

Verify

- pnpm --filter @remotedesk/shared build && pnpm -r test
- Manual: POST a valid receipt -> stored with a signature; GET /receipts/:id/verify -> { valid:true, signed:true, signatureValid:true }; fetch the public key and verify offline with verifyReceiptSignature.

FILE: [generated-pack26-receipt-signing-code-review.md](#)

Pack 26 — Code Review Notes

Strengths

- Real non-repudiation: detached Ed25519 over a canonical, content-free preimage that binds org + receiptId + rootHash + issuedAt.
- Offline verifiable: shared verifyReceiptSignature + published public key, no server dependency.
- Private key isolation: only the repo's signingPrivate/ensureKey select it; only the signer service touches it; never serialized.
- Uses Node built-in crypto (no new deps); base64url raw keys.
- Signs only AFTER Pack 24 validation, so signature + chain always agree.
- Rotation-aware via keyId + publicKeyById.

Reviewer checklist

- [] No route or log emits SigningKey.privateKey (grep privateKey).
- [] store persists the signature columns; getJson returns receipt.signature.
- [] storeVerified signs after validateReceipt success only.
- [] verify route loads the key by signature.keyId (post-rotation safe).
- [] base64url round-trips for keys + signature.

Safety

- Tampering with org, receiptId, rootHash, or issuedAt invalidates the signature.
- Content-free preimage; no session content is signed or exposed.
- Private key never leaves the server.

FILE: [generated-pack26-receipt-signing-manifest.json](#)

```
{
  "pack": "pack26-receipt-signing",
  "title": "Ed25519 Receipt Signing + Server Key Management for Non-Repudiation",
  "actualFileCount": 15,
  "filesCreated": [
    "packages/shared/src/permissions/receiptSignature.ts",
    "apps/api/prisma/migrations/0026_signing_keys/migration.sql",
    "apps/api/src/lib/ed25519.ts",
    "apps/api/src/repositories/signingKeyRepository.ts",
    "apps/api/src/services/admin/receiptSigner.service.ts",
    "apps/web/app/dashboard/receipts/SignatureBadge.tsx",
    "tests/admin/receiptSignature.test.ts",
    "tests/admin/ed25519.test.ts",
    "tests/admin/receiptSigner.service.test.ts"
  ],
  "filesModified": [
    "packages/shared/src/permissions/index.ts",
    "packages/shared/src/permissions/consentReceipt.ts",
```

```

    "apps/api/prisma/schema.prisma",
    "apps/api/src/services/admin/receiptVerifier.service.ts",
    "apps/api/src/routes/deviceAdmin.routes.ts",
    "apps/web/app/dashboard/receipts/page.tsx"
  ],
  "filesNotTouched": [
    "apps/desktop/**",
    "apps/api/src/lib/nodeSha256.ts"
  ],
  "dependenciesRequired": {
    "new": [],
    "existing": ["@remotedesk/shared", "@prisma/client", "node:crypto", "express", "next", "react", "vitest"],
    "assumedFromProject": [
      "Pack 22 ConsentReceipt + rootHash",
      "Pack 23 consentReceiptRepository (store/getJson)",
      "Pack 24 receiptVerifier validateReceipt",
      "orgContext providing orgId"
    ]
  },
  "knownLimitations": [
    "One active signing key per org (rotation supported via keyId; rotation UI out of scope).",
    "Private key stored at rest in the DB column; wrap with a KMS/HSM for production hardening (follow-up).",
    "Signature binds the rootHash, which commits to the timeline; it does not separately sign each line.",
    "Desktop does not verify signatures yet (server + auditors do); a desktop verify is a small follow-up."
  ],
  "remainingManualWiringRisks": [
    "Persist + return signature columns in consentReceiptRepository.",
    "Sign only after validateReceipt passes in storeVerified.",
    "Ensure no route/log exposes the private key."
  ],
  "estimatedCompletionAfterCarefulMerge": "82-87%"
}

```

Pack 26 done: 15 files (9 created, 6 modified). Consent receipts now carry real cryptographic non-repudiation. After the Pack 24 verifier confirms a receipt is valid and content-free, the server signs its canonical digest (org + receiptId + rootHash + issuedAt) with an Ed25519 key and publishes the public key, so any auditor can verify a receipt entirely offline with the shared verifier, no server call and no private material.

The clean bit: the signature binds the org, so a valid receipt can't be silently re-attributed to another tenant; binds the rootHash, which already commits to the whole Pack 22 timeline, so one signature covers everything; and uses Node's built-in Ed25519 (no new deps). The private key is rigorously isolated, selected only by the key repo's internal methods, used only inside the signer service, and never serialized to a response or log (there's even a test asserting it can't leak onto the signature object). Signing happens strictly after validation, so the signature and the hash chain can never disagree.

That takes the receipt system from "internally consistent" (22), to "verified + content-free" (24), to "cryptographically attributable" (26). The one production-hardening follow-up flagged: the private key sits in a DB column, wrap it with a KMS/HSM before going live. Next natural slice = key rotation UI + KMS envelope, or a desktop-side signature verify so the host can confirm its own receipts. Bolo.

